

[linear.algebra]R0

A proposal to add linear algebra support to the C++ standard library

Draft Proposal, 2019-01-21

This version:

https://github.com/BobSteagall/linear_algebra_private/papers/P1385.bs

Latest published version:

<http://wg21.link/P1385>

Authors:

[Guy Davidson](#)

[Bob Steagall](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This document proposes an interface specification for linear algebra facilities in the standard C++ library.

Table of Contents

- 1 **Introduction**
- 2 **Goals**
- 3 **Definitions**
 - 3.1 Mathematical Terms
 - 3.2 Terms Pertaining to C++ Types
 - 3.3 Overloaded Terms

- 3.3.1 Matrix
- 3.3.2 Vector
- 3.3.3 Dimension
- 3.3.4 Rank

4 Scope

- 4.1 Functionality
- 4.2 Considered but Excluded

5 Design Aspects

- 5.1 Memory Source
- 5.2 Addressing Model
- 5.3 Memory Ownership
- 5.4 Capacity and Resizability
- 5.5 Element Layout
- 5.6 Element Access and Indexing
- 5.7 Element Type
- 5.8 Mixed-Element-Type Expressions
- 5.9 Mixed-Engine Expressions
- 5.10 Concurrency and Parallelism
- 5.11 Linear Algebra and constexpr

6 Interface Description

- 6.1 Engine Types and Supporting Traits
 - 6.1.1 Fixed-Size Engine
 - 6.1.2 Dynamically-Resizable Engine
 - 6.1.3 Transpose Engine
 - 6.1.4 Element Promotion Traits
 - 6.1.5 Engine Promotion Traits for Negation
 - 6.1.6 Engine Promotion Traits for Addition
 - 6.1.7 Engine Promotion Traits for Subtraction
 - 6.1.8 Engine Promotion Traits for Multiplication
- 6.2 Mathematical Types
 - 6.2.1 Helpers
 - 6.2.2 Column Vector
 - 6.2.3 Row Vector

6.2.4	Matrix
6.3	Matrix Operation Traits
6.3.1	Negation Traits
6.3.2	Addition Traits
6.3.3	Subtraction Traits
6.3.4	Multiplication Traits
6.4	Arithmetic Operators
6.4.1	Negation Operator
6.4.2	Addition Operator
6.4.3	Subtraction Operator
6.4.4	Multiplication Operator
6.5	Type Aliases

§ 1. Introduction

Linear algebra is a mathematical discipline of ever-increasing importance, with direct application to a wide variety of problem domains, such as signal processing, computer graphics, medical imaging, scientific simulations, machine learning, analytics, financial modeling, and high-performance computing. And yet, despite the relevance of linear algebra to so many aspects of modern computing, the C++ standard library does not include a set of linear algebra facilities. This paper proposes to remedy this deficit for C++23.

This paper should be read after P1166, in which we describe a high-level set of expectations for what a linear algebra library should contain.

§ 2. Goals

We expect that typical users of a standard linear algebra library are likely to value two features above all else: ease-of-use (including expressiveness), and high performance. This set of users will expect the ability to compose arithmetical expressions of linear algebra objects similarly to what one might find in a textbook; indeed, this has been deemed a "must-have" feature by several participants in recent SG14 Linear Algebra SIG conference calls. And for a given arithmetical expression, they will expect run-time computational performance that is close to what they could obtain with an equivalent sequence of function calls to a more "traditional" linear algebra library, such as *LAPCK*, *Blaze*, *Eigen*, etc.

There also exists a set of linear algebra “super-users” who will value most highly a third feature – the ability to customize underlying infrastructure in order to maximize performance for specific problems and computing platforms. These users seek the highest possible run-time performance, and to achieve it, require the ability to customize any and every portion of the library’s computational infrastructure.

With these high-level user requirements in mind, we propose an interface specification intended to achieve the following goals:

1. To provide a set of vocabulary types for representing the mathematical objects and operations that are relevant to linear algebra;
2. To provide a public interface for linear algebra operations that is intuitive, teachable, and mimics the expressiveness of mathematical notation to the greatest *reasonable* extent;
3. To exhibit out-of-the-box performance rivalling that of an equivalent sequence of function calls to a more traditional linear algebra library, such as *LAPACK*, *Blaze*, *Eigen*, etc.;
4. To provide a set of building blocks that manage the source, ownership, lifetime, layout, and access to the memory required to represent the linear algebra vocabulary types, with the requirement that these building blocks are also suitable for eventually representing other interesting mathematical entities, such as quaternions, octonions, and tensors; and,
5. To provide straightforward facilities and techniques for customization that enable users to optimize performance for their specific problem domain on their specific hardware.

§ 3. Definitions

When discussing linear algebra and related topics for a proposal such as this, it is important to note that there are several overloaded terms (such as *matrix*, *vector*, *dimension*, and *rank*) which must be defined and disambiguated if such discussions are to be productive. These terms have specific meanings in mathematics, as well as different, but confusingly similar, meanings to C++ programmers.

In the following sections we provide definitions for relevant mathematical concepts, C++ type design concepts, and describe how this proposal employs those overloaded terms in various contexts.

§ 3.1. Mathematical Terms

In order to facilitate subsequent discussion, we first provide the following *informal* set of definitions for important mathematical concepts:

1. A **vector space** is a collection of **vectors**, where vectors are objects that may be added together and multiplied by scalars. Euclidean vectors are an example of a vector space, typically used to represent displacements, as well as physical quantities such as force or momentum. Linear algebra is concerned primarily with the study of vector spaces.
2. The **dimension** of a vector space is defined as the minimum number of coordinates required to specify any point within the space.
3. A **matrix** is a rectangular array of numbers, symbols, or expressions, arranged in rows and columns. A matrix having m rows and n columns is said to have size $m \times n$. Although matrices can be used solve systems of simultaneous linear equations, they are most commonly used to represent linear transformations, solve linear least squares problems, and to explore and/or manipulate the properties of vector spaces.
4. The **rank** of a matrix is the dimension of the vector space spanned by its columns, which is equal to the dimension of the vector space spanned by its rows. The rank is also equal to the maximum number of linearly-independent columns and rows.
5. An **element** of a matrix is an individual member (number, symbol, expression) of the rectangular array comprising the matrix, lying at the intersection of a single row and a single column. In traditional mathematical notation, row and column indexing is 1-based, where rows are indexed from 1 to m and columns are indexed from 1 to n . Given some matrix A , element a_{11} refers to the element in the upper left-hand corner of the array and element a_{mn} refers to the element in the lower right-hand corner.
6. A **row vector** is a matrix containing a single row; in other words, a matrix of size $1 \times n$. In many applications of linear algebra, row vectors represent spatial vectors.
7. A **column vector** is a matrix containing a single column; in other words, a matrix of size $m \times 1$. In many applications of linear algebra, column vectors represent spatial vectors.
8. **Element transforms** are non-arithmetical operations that modify the relative positions of elements in a matrix, such as transpose, column exchange, and row exchange.
9. **Element arithmetic** refers to arithmetical operations that read or modify the values of individual elements independently of other elements, such as assigning a value to a specific element or multiplying a row by some value.
10. **Matrix arithmetic** refers to the assignment, addition, subtraction, negation, multiplication, and determinant operations defined for matrices, row vectors, and column vectors as wholes.
11. A **rectangular matrix** is a matrix requiring a full $m \times n$ representation; that is, a matrix not possessing a special form, such as identity, triangular, band, etc.
12. The **identity matrix** is a square matrix where all elements on the diagonal are equal to one and all off-diagonal elements are equal to zero.

13. A **triangular matrix** is a matrix where all elements above or below the diagonal are zero; those with non-zero elements above the diagonal are called *upper triangular*, while those with non-zero elements below the diagonal are called *lower triangular*.
14. A **band matrix** is a sparse matrix whose non-zero entries are confined to a diagonal band, lying on the main diagonal and zero or more diagonals on either side.
15. **Decompositions** are complex sequences of arithmetic operations, element arithmetic, and element transforms performed upon a matrix that expose important mathematical properties of that matrix. Several types of decomposition are often performed in solving least-squares problems.
16. **Eigen-decompositions** are decompositions performed upon a symmetric matrix in order to compute the eigenvalues and eigenvectors of that matrix; this is often performed when solving problems involving linear dynamic systems.

§ 3.2. Terms Pertaining to C++ Types

The following are terms used in this proposal that describe various aspects of how the mathematical concepts described above in Section 3.1 might be implemented:

1. An **array** is a data structure representing an indexable collection of objects (elements) such that each element is identified by at least one index. An array is said to be *one-dimensional* array if its elements are accessible with a single index; a *multi-dimensional* array is an array for which more than one index is required to access its elements.
2. The **dimension** of an array refers to the number of indices required to access an element of that array. The **rank** of an array is a synonym for its dimension.
3. This proposal uses the term **MathObj** to refer generically to one of the C++ types described herein representing matrices, row vectors, and column vectors. These are the public-facing facilities developers will use in their code.
4. An **engine** is an implementation type that manages the storage-related aspects of, and access to, the elements of a *MathObj*. In this proposal, an engine object is a private member of a *MathObj*. Other than as a template parameter, engines are not part of a *MathObj*'s public interface.
5. The adjective **dense** refers to a *MathObj* representation where storage is allocated for every element.
6. The adjective **sparse** refers to a *MathObj* representation where storage is allocated only for non-zero elements;
7. **Storage** is used by this proposal as a synonym for memory.

8. **Traits** refers to a stateless class template that provides some set of services, normalizing those services over its set of template parameters.
9. **Row size** and **column size** refer to the number of rows and columns, respectively, that a *MathObj* represents, which must be less than or equal to its row and column capacities, defined below.
10. **Row capacity** and **column capacity** refer to the maximum number of rows and columns, respectively, that a *MathObj* can possibly represent.
11. **Fixed-size** (FS) refers to an engine type whose row and column sizes are fixed at instantiation time and constant thereafter.
12. **Fixed-capacity** (FC) refers to an engine type whose row and column capacities are fixed at instantiation time and constant thereafter.
13. **Dynamically re-sizable** (DR) refers to an engine type whose row and column sizes and capacities may be changed at run time.

§ 3.3. Overloaded Terms

This section describes how we use certain overloaded terms in this proposal.

§ 3.3.1. Matrix

The term *matrix* is frequently used by C++ programmers to mean a general-purpose array of arbitrary size. For example, one of the authors worked at a company where it was common practice to refer to 4-dimensional arrays as “4-dimensional matrices.”

In this proposal, we use the word *array* only to mean a data structure whose elements are accessible using one or more indices, and which has no invariants pertaining to higher-level or mathematical meaning.

We use *matrix* to mean the mathematical object as defined above in Section 3.1, and `matrix` (in monospaced font) to mean the C++ class template that implements the mathematical object. We sometimes use `MathObj` (in monospaced font) in some of the component interface code and descriptions below to generically refer to `matrix`, `row_vector`, or `column_vector`.

§ 3.3.2. Vector

Likewise, many C++ programmers incorrectly use the term *vector* as a synonym for “dynamically re-sizable array.” This bad habit is exacerbated by the unforgivably awful naming of `std::vector`.

This proposal uses the term *vector* to mean an element of a vector space, per Section 3.1. Further, *row vector* and *column vector* have the meanings set out in 3.1, while `row_vector` and `column_vector` (in monospaced font) are the C++ class templates implementing those mathematical objects, respectively. We sometimes use `MathObj` (in monospaced font) in some of the component code descriptions below to generically refer to `row_vector`, `column_vector`, or `matrix`.

§ 3.3.3. Dimension

In linear algebra, a vector space V is said to be of *dimension* n , or *n-dimensional*, if there exist n linearly independent vectors which span V . This is another way of saying that n is the minimum number of coordinates required to specify any point in V . However, in common programming parlance, *dimension* refers to the number of indices used to access an element in an array.

We use the term dimension both ways in this proposal but try to do so consistently and in a way that is clear from the context. For example, a matrix describing a rotation in a 3D virtual reality application is an example of a 2-dimensional data structure containing 3-dimensional row and column vectors. A vector describing an electric field is an example of a 1-dimensional data structure implemented as a 3-dimensional row vector.

§ 3.3.4. Rank

The *rank* of a matrix is the dimension of the vector space spanned by its columns (or rows), which corresponds to the maximal number of linearly independent columns (or rows) of that matrix. Rank also has yet another meaning in tensor analysis, where it is commonly used as a synonym for a tensor's *order*.

However, rank also has a meaning in computer science where it is used as a synonym for dimension. In the C++ standard at `[meta.unary.prop.query]`, rank is described as the number of dimensions of `T` if `T` names an array, otherwise it is zero.

We avoid using the term *rank* in this proposal in the context of linear algebra, except as a quantity that might result from performing certain decompositions.

§ 4. Scope

We contend that the best approach for standardizing a set of linear algebra components for C++23 will be one that is layered, iterative, and incremental. This paper is quite deliberately a “linear algebra-only” proposal; it describes what we believe is the minimum set of components necessary to provide a

certain basic level of functionality. Higher-level functionality can be specified in terms of the interfaces described here, and we encourage succession papers to explore this possibility.

§ 4.1. Functionality

The foundational layer, as described here, should include the minimal set of types and functions required to perform matrix functions in finite dimensional spaces. This includes:

- Matrix, row vector, and column vector class templates;
- Arithmetic operations for addition, subtraction, negation, and multiplication of matrices and vectors;
- Arithmetic operations for scalar multiplication of matrices and vectors;
- Well-defined facilities for creating custom engines; and,
- Customization points for optimizing the performance of transform and arithmetical operations.

§ 4.2. Considered but Excluded

Tensors

There has been a great deal of interest expressed in specifying an interface for general-purpose tensor processing in which linear algebra facilities fall out as a special case. We exclude this idea from this proposal for two reasons. First, given the practical realities of standardization work, the enormous scope of such an effort would very likely delay introduction of linear algebra facilities until C++26 or later.

Second, and more importantly, implementing matrices as derived types or specializations of a general-purpose tensor type is bad type design. Consider the following: a tensor is (informally) an array of mathematical objects (numbers or functions) such that its elements transform according to certain rules under a coordinate system change. In a p -dimensional space, a tensor of rank n will have p^n elements. In particular, a rank-2 tensor in a p -dimensional space may be represented by a $p \times p$ matrix having certain properties related to coordinate transformation not possessed by all $p \times p$ matrices.

These defining characteristics of a tensor lead us to the crux of the issue: every rank-2 tensor can be represented by a square matrix, but not every square matrix represents a tensor. As one quickly realizes, only a small fraction of all possible matrices are representations of rank-2 tensors.

All of this is a long way of saying that the class invariants governing a matrix type are quite different from those governing a tensor type, and as such, the public interfaces of such types will also differ

substantially.

From this we conclude that matrices are not Liskov-substitutable for rank-2 tensors, and therefore as matter of good type design, matrices and tensors should be implemented as distinct types, perhaps with appropriate inter-conversion operations.

This situation is analogous to the age-old object-oriented design question: when designing a group of classes that represent geometric shape, is a square a kind of rectangle? In other words, should class `square` be publicly derived from class `rectangle`? Mathematically, yes, a square *is* a rectangle. But from the perspective of good interface design, class `square` is not substitutable for class `rectangle` and is usually best implemented as a distinct type having no IS-A relationship with `rectangle`.

Quaternions and Octonions

There has also been interest expressed in including other useful mathematical objects, such as quaternions and octonions, as part of a standard linear algebra library. Although element storage for these types might be implemented using the engines described in this proposal, quaternions and octonions represent mathematical concepts that are fundamentally different from those of matrices and vectors.

As with tensors, the class invariants and public interfaces for quaternions and octonions would be substantially different from that of the linear algebra components. Liskov substitutability would not be possible, and therefore quaternions and octonions should be implemented as a set of types distinct from the linear algebra types.

§ 5. Design Aspects

The following describe several important aspects of the problem domain affecting the design of the proposed interface. Importantly, these aspects are orthogonal, and are addressable through judicious combinations of template parameters and implementation type design.

§ 5.1. Memory Source

Perhaps the first question to be answered is that of the source of memory in which elements will reside. One can easily imagine multiple sources of memory:

- Elements reside in an external buffer allocated from the global heap.
- Elements reside in an external buffer allocated by a custom allocator and/or specialized heap.

- Elements reside in an external fixed-size buffer that exists independently of the *MathObj*, not allocated from a heap, and which has a lifetime greater than that of the *MathObj*.
- Elements reside in a fixed-size buffer that is a member of the *MathObj* itself.
- Elements reside collectively in a set of buffers distributed across multiple machines.

§ 5.2. Addressing Model

It is also possible that the memory used by a *MathObj* might be addressed using what the standard calls a *pointer-like type*, also known as a *fancy pointer*.

For example, consider an element buffer existing in a shared memory segment managed by a custom allocator. In this case, the allocator might employ a fancy pointer type that performs location-independent addressing based on a segment index and an offset into that segment.

One can also imagine a fancy pointer that is a handle to a memory resource existing somewhere on a network, and addressing operations require first mapping that resource into the local address space, perhaps by copying over the network or by some magic sequence of RPC invocations.

§ 5.3. Memory Ownership

The next important questions pertain to memory ownership. Should the memory in which elements reside be deallocated, and if so, what object is responsible for performing the deallocation?

A *MathObj* might own the memory in which it stores its elements, or it might employ some non-owning view type, like `mdspan`, to manipulate elements owned by some other object.

§ 5.4. Capacity and Resizability

As with `std::string` and `std::vector`, it is occasionally useful for a *MathObj* to have excess storage capacity in order to reduce the number of re-allocations required by future resizing operations. Some linear algebra libraries, like *LAPACK*, account for the fact that a *MathObj*'s capacity may be different than its size. This capability was of critical importance to the success of one author's prior work in functional MRI image analysis.

In other problem domains, like computer graphics, *MathObjs* are small and always of the same size. In this case, the size and capacity are equal, and there is no need for a *MathObj* to maintain or manage excess capacity.

§ 5.5. Element Layout

There are many ways to arrange the elements of a matrix in memory, the most common in C++ being row-major dense rectangular. In Fortran-based libraries, the two-dimensional arrays used to represent matrices are usually column-major. There are also special arrangements of elements for upper/lower triangular and banded diagonal matrices that are both row-major and column-major. These arrangements of elements have been well-known for many years, and libraries like LAPACK in the hands of a knowledgeable user can use them to implement code that is optimal in both time and space.

§ 5.6. Element Access and Indexing

In keeping with the goal of supporting a natural syntax, and in analogy with the indexing operations provided by the random-access standard library containers, it seems reasonable to provide both const and non-const indexing for reading and writing individual elements.

However, support for element indexing raises an important question: should *MathObjs* employ 1-based indexing or 0-based indexing? 1-based indexing is the convention used in mathematical notation (and Fortran), whereas 0-based indexing is “the C++ way.”

§ 5.7. Element Type

C++ supports a relatively narrow range of arithmetic types, lacking direct support for arbitrary precision numbers and fixed-point numbers, among others. Libraries exist to implement these types, and they should not be precluded from use in a standard linear algebra library. It is possible that individual elements of a *MathObj* may allocate memory, and therefore an implementation cannot assume that element types have trivial constructors or destructors.

§ 5.8. Mixed-Element-Type Expressions

In general, when multiple built-in arithmetic types are present in an arithmetical expression, the resulting type will have a precision greater than or equal to that of the type with greatest precision in the expression. In other words, to the greatest reasonable extent, information is preserved.

A similar principal should apply to expressions involving *MathObjs* where more than one element type is present. Arithmetic operations involving *MathObjs* should, to the greatest reasonable extent, preserve element-wise information.

For example, just as the result of multiplying a `float` by a `double` is a `double`, the result multiplying a matrix-of-`float` by a matrix-of-`double` should be a matrix-of-`double`. We call the process of determining the resulting element type *element promotion*.

§ 5.9. Mixed-Engine Expressions

In analogy with element type, *MathObj* expressions may include mixed storage management strategies as implemented by their corresponding engine types. For example, consider the case of a fixed-size matrix multiplied by a dynamically-resizable matrix. What is the engine type of the resulting matrix?

Expression involving mixed engine types should not limit the availability of basic arithmetic operations. This means that there should be a mechanism for determining the engine type of the resulting from such expressions. We call the process of determining the resulting engine type *engine promotion*.

§ 5.10. Concurrency and Parallelism

In pursuit of optimal performance, developers may want to use multiple cores to carry out multiplication on very large pairs of matrices, particularly in situations where the operation is used to produce a third matrix rather than modify one of the operands. The matrix multiplication operation is highly amenable to this approach, since a thread may be used for each row of the source matrix.

Developers may also wish to make use of SIMD intrinsics to enable parallel evaluation of matrix multiplication. This is common in game development environments where programs are written for very specific platforms, where the make and model of processor is well defined. This would impact on element layout and storage. Such work has already been demonstrated in paper N4454.

§ 5.11. Linear Algebra and `constexpr`

The fundamental set of operations for linear algebra can all be implemented in terms of a subset of the algorithms defined in the `<algorithm>` header, all of which are marked `constexpr` since C++20. Matrix and vector initialization is of course also possible at compile time.

The arrival of `std::is_constant_evaluated` in C++20 makes it possible to offer a `constexpr` implementation of the operations, allowing customizations to defer to them in `constexpr` evaluations while taking the customization's notionally superior run-time path in alternate situations.

§ 6. Interface Description

In this section, we describe the various types, operators, and functions comprising the proposed interface. The reader should note that the descriptions below are by no means ready for wording; rather, they are intended to foster further discussions and refinements.

§ 6.1. Engine Types and Supporting Traits

All of the engine types provide a basic interface for const element indexing, row and column sizes, and row and column capacities. They also export public type aliases which specify their element type, whether or not they are resizable, and a 2-tuple for containing sizes and capacities.

§ 6.1.1. Fixed-Size Engine

```
template<class T, size_t R, size_t C>
class fs_matrix_engine
{
    static_assert(is_matrix_element_v<T>);
    static_assert(R >= 1);
    static_assert(C >= 1);

public:
    using element_type      = T;
    using is_resizable_type = false_type;
    using size_tuple        = tuple<size_t, size_t>;

public:
    fs_matrix_engine();
    fs_matrix_engine(fs_matrix_engine&&);
    fs_matrix_engine(fs_matrix_engine const&);

    fs_matrix_engine& operator =(fs_matrix_engine&&);
    fs_matrix_engine& operator =(fs_matrix_engine const&);

    T          operator ()(size_t i) const;
    T          operator ()(size_t i, size_t j) const;
    T const*   data() const;

    size_t     columns() const noexcept;
    size_t     rows() const noexcept;
    size_tuple size() const noexcept;

    size_t     column_capacity() const noexcept;
    size_t     row_capacity() const noexcept;
    size_tuple capacity() const noexcept;

    T&         operator ()(size_t i);
    T&         operator ()(size_t i, size_t j);
    T*         data();

    void        swap_columns(size_t i, size_t j);
    void        swap_rows(size_t i, size_t j);

private:
    T          ma_elems[R*C]; //- for exposition
```

```
    T*      mp_bias;          //- bias pointer for 1-based indexing; for expo  
};
```

Class template `fs_matrix_engine<T, R, C>` implements a fixed-size, fixed-capacity engine. In addition to the basic engine interface, it provides member functions for mutable element indexing and swapping rows and/or columns.

§ 6.1.2. Dynamically-Resizable Engine

```
template<class T, class ALLOC = std::allocator<T>>
class dyn_matrix_engine
{
    static_assert(is_matrix_element_v<T>);

public:
    using element_type      = T;
    using is_resizable_type = true_type;
    using size_tuple        = tuple<size_t, size_t>;

public:
    dyn_matrix_engine();
    dyn_matrix_engine(dyn_matrix_engine&&);
    dyn_matrix_engine(dyn_matrix_engine const&);

    dyn_matrix_engine& operator =(dyn_matrix_engine&&);
    dyn_matrix_engine& operator =(dyn_matrix_engine const&);

    T          operator ()(size_t i) const;
    T          operator ()(size_t i, size_t j) const;
    T const*   data() const;

    size_t     columns() const noexcept;
    size_t     rows() const noexcept;
    size_tuple size() const noexcept;

    size_t     column_capacity() const noexcept;
    size_t     row_capacity() const noexcept;
    size_tuple capacity() const noexcept;

    T&         operator ()(size_t i);
    T&         operator ()(size_t i, size_t j);
    T*         data();

    void        reserve(size_tuple cap);
    void        reserve(size_t rowcap, size_t colcap);

    void        resize(size_tuple size);
    void        resize(size_t rows, size_t cols);
    void        resize(size_tuple size, size_tuple cap);
    void        resize(size_t rows, size_t cols, size_t rowcap, size_t colcap);
```

```

    void    swap_columns(size_t i, size_t j);
    void    swap_rows(size_t i, size_t j);

private:
    using pointer = typename std::allocator_traits<ALLOC>::pointer;

    pointer    mp_elems;    //- for exposition
    T*         mp_bias;     //- bias pointer for 1-based indexing; for expo
    size_t     m_rows;
    size_t     m_cols;
    size_t     m_rowcap;
    size_t     m_colcap;
};

```

Class template `dyn_matrix_engine<T, ALLOC>` implements an engine whose sizes and capacities can be changed at runtime. In addition to the basic engine interface, it provides member functions for mutable element indexing, changing size and capacity, and swapping rows and/or columns.

§ 6.1.3. Transpose Engine

```
template<class ENG>
class matrix_transpose_engine
{
public:
    using engine_type      = ENG;
    using element_type     = typename engine_type::element_type;
    using is_resizable_type = false_type;
    using size_tuple       = typename engine_type::size_tuple;

public:
    matrix_transpose_engine();
    matrix_transpose_engine(engine_type const& eng);
    matrix_transpose_engine(matrix_transpose_engine&&);
    matrix_transpose_engine(matrix_transpose_engine const&);

    matrix_transpose_engine& operator =(matrix_transpose_engine&&);
    matrix_transpose_engine& operator =(matrix_transpose_engine const&);

    element_type      operator ()(size_t i) const;
    element_type      operator ()(size_t i, size_t j) const;
    element_type const* data() const;

    size_t      columns() const noexcept;
    size_t      rows() const noexcept;
    size_tuple  size() const noexcept;

    size_t      column_capacity() const noexcept;
    size_t      row_capacity() const noexcept;
    size_tuple  capacity() const noexcept;

private:
    engine_type* mp_other;
};
```

Class template `matrix_transpose_engine<ENG>` implements a non-owning, const view type that provides the basic engine interface. Its primary use is as the return value of the `tr()` member function of the *MathObj* types.

§ 6.1.4. Element Promotion Traits

```
template<class T> struct is_complex;
template<class T> constexpr bool is_complex_v = ...;
```

Traits type `is_complex<T>` determines whether its template argument `T` is of type `std::complex<V>` for some type `V`, where `V` must itself be an arithmetical type as determined by `std::is_arithmetic_v<U>`. Defining what constitutes an arithmetic type can be challenging; our intention is that an arithmetic type is one representing a field.

```
template<class T> struct is_matrix_element;
template<class T> constexpr bool is_matrix_element_v = ...;
```

Traits type `is_matrix_element<T>` is used in static assertions to ensure that *MathObj* types are instantiated only with element types representing a field (i.e., arithmetic types, or complex types per above). It uses `is_complex<T>` to help make that determination.

§ 6.1.5. Engine Promotion Traits for Negation

```
template<class E1>
struct matrix_engine_negate_promotion
{
    using engine_type = ...;
};

template<class E1>
using matrix_engine_negate_t = typename matrix_engine_negate_promotion<E1>::
```

Class template `matrix_engine_negate_promotion<E1>` implements a traits type that determines the resulting engine type when negating a *MathObj*. It is used by `matrix_negation_traits<E1>` as the return value to the

§ 6.1.6. Engine Promotion Traits for Addition

```
template<class E1, class E2>
struct matrix_engine_add_promotion
{
    using engine_type = ...;
};

template<class E1, class E2>
using matrix_engine_add_t = typename matrix_engine_add_promotion<E1, E2>::engine_type;
```

Class template `matrix_engine_add_promotion<E1, E2>` implements a traits type that determines the resulting engine type when adding two compatible *MathObjs*.

§ 6.1.7. Engine Promotion Traits for Subtraction

```
template<class E1, class E2>
struct matrix_engine_subtract_promotion
{
    using engine_type = ...;
};

template<class E1, class E2>
using matrix_engine_subtract_t = typename matrix_engine_subtract_promotion<E1, E2>::engine_type;
```

Class template `matrix_engine_subtract_promotion<E1, E2>` implements a traits type that determines the resulting engine type when subtracting two compatible *MathObjs*.

§ 6.1.8. Engine Promotion Traits for Multiplication

```
template<class E1, class E2>

struct matrix_engine_multiply_promotion;

{
    using engine_type = ...;
};

template<class E1, class E2>
using matrix_engine_multiply_t = typename matrix_engine_multiply_promotion<
```

Class template `matrix_engine_multiply_promotion<E1, E2>` implements a traits type that determines the resulting engine type when multiplying two compatible *MathObjs*.

§ 6.2. Mathematical Types

This section describes the three main linear algebra object types proposed herein: the class templates `column_vector`, `row_vector`, and `matrix`.

§ 6.2.1. Helpers

```
template<class ET1, class ET2>
using enable_if_resizable_t =
    typename std::enable_if_t<is_same_v<ET1, ET2> && ET1::is_resizable_type
```

Alias template `enable_if_resizable_t<ET1, ET2>` is a helper used by the *MathObj* types to manipulate overload resolution sets for member functions that perform dynamic storage management.

§ 6.2.2. Column Vector

```
template<class ENG>
class column_vector
{
public:
    using engine_type      = ENG;
    using element_type     = typename ENG::element_type;
    using is_resizable_type = typename ENG::is_resizable_type;
    using size_tuple       = typename ENG::size_tuple;
    using transpose_type    = row_vector<matrix_transpose_engine<ENG>>;

public:
    ~column_vector() = default;
    column_vector();
    column_vector(column_vector&&) = default;
    column_vector(column_vector const&) = default;

    template<class ET2>
    column_vector(column_vector<ET2> const& src);

    template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
    column_vector(size_t rows);
    template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
    column_vector(size_t rows, size_t rowcap);

    column_vector& operator =(column_vector&&) = default;
    column_vector& operator =(column_vector const&) = default;

    template<class ET2>
    column_vector& operator =(column_vector<ET2> const& rhs);

    //- Const element access.
    //
    element_type      operator ()(size_t i) const;
    element_type const* data() const;

    //- Accessors.
    //
    size_t  columns() const noexcept;
    size_t  rows() const noexcept;
    size_t  size() const noexcept;
```

```

size_t    column_capacity() const noexcept;
size_t    row_capacity() const noexcept;
size_t    capacity() const noexcept;

//- Common functions.
//
transpose_type  tr() const;

//- Mutable element access.
//
element_type    operator ()(size_t i);
element_type*    data();

//- Change capacity.
//
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void    reserve(size_t rowcap);

//- Change size.
//
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void    resize(size_t rows);

//- Change size and capacity in one shot.
//
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void    resize(size_t rows, size_t rowcap);

//- Row operations.
//
void    swap_rows(size_t i, size_t j);

private:
    engine_type    m_engine;    //- for exposition
};

```

Class template `column_vector<ENG>` provides a representation of a column vector, with element type and storage management implemented by the engine type `ENG`. If the engine provides resizing, then this class will as well.

§ 6.2.3. Row Vector

```
template<class ENG>
class row_vector
{
public:
    using engine_type      = ENG;
    using element_type     = typename ENG::element_type;
    using is_resizable_type = typename ENG::is_resizable_type;
    using size_tuple       = typename ENG::size_tuple;
    using transpose_type    = column_vector<matrix_transpose_engine<ENG>>;

public:
    ~row_vector() = default;
    row_vector();
    row_vector(row_vector&&) = default;
    row_vector(row_vector const&) = default;

    template<class ET2>
    row_vector(row_vector<ET2> const& src);

    template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
    row_vector(size_t cols);
    template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
    row_vector(size_t cols, size_t colcap);

    row_vector& operator =(row_vector&&) = default;
    row_vector& operator =(row_vector const&) = default;

    template<class ET2>
    row_vector& operator =(row_vector<ET2> const& rhs);

    //- Const element access.
    //
    element_type      operator ()(size_t i) const;
    element_type const* data() const;

    //- Accessors.
    //
    size_t  columns() const noexcept;
    size_t  rows() const noexcept;
    size_t  size() const noexcept;
```

```

size_t    column_capacity() const noexcept;
size_t    row_capacity() const noexcept;
size_t    capacity() const noexcept;

//- Common functions.
//
transpose_type  tr() const;

//- Mutable element access.
//
element_type    operator ()(size_t i);
element_type*    data();

//- Change capacity.
//
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void    reserve(size_t colcap);

//- Change size.
//
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void    resize(size_t cols);

//- Change size and capacity in one shot.
//
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void    resize(size_t cols, size_t colcap);

//- column operations.
//
void    swap_columns(size_t i, size_t j);

private:
    engine_type    m_engine;    //- for exposition
};

```

Class template `row_vector<ENG>` provides a representation of a row vector, with element type and storage management implemented by the engine type `ENG`. If the engine provides resizing, then this class will as well.

§ 6.2.4. Matrix

```
template<class ENG>
class matrix
{
public:
    using engine_type      = ENG;
    using element_type     = typename ENG::element_type;
    using is_resizable_type = typename ENG::is_resizable_type;
    using size_tuple       = typename ENG::size_tuple;
    using transpose_type    = matrix<matrix_transpose_engine<ENG>>;

public:
    ~matrix() = default;
    matrix();
    matrix(matrix&&) = default;
    matrix(matrix const&) = default;

    template<class ET2>
    matrix(matrix<ET2> const& src);

    template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
    matrix(size_tuple size);
    template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
    matrix(size_t rows, size_t cols);
    template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
    matrix(size_t rows, size_t cols, size_t rowcap, size_t colcap);

    matrix&      operator =(matrix&&) = default;
    matrix&      operator =(matrix const&) = default;

    template<class ET2>
    matrix&      operator =(matrix<ET2> const& rhs);

    //- Const element access.
    //
    element_type      operator ()(size_t i, size_t j) const;
    element_type const* data() const;

    //- Accessors.
    //
    size_t      columns() const noexcept;
    size_t      rows() const noexcept;
```

```

size_tuple size() const noexcept;

size_t      column_capacity() const noexcept;
size_t      row_capacity() const noexcept;
size_tuple  capacity() const noexcept;

//- Common functions.
//
transpose_type tr() const;

//- Mutable element access.
//
element_type operator()(size_t i, size_t j);
element_type* data();

//- Change capacity.
//
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void reserve(size_tuple cap);
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void reserve(size_t rowcap, size_t colcap);

//- Change size.
//
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void resize(size_tuple size);
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void resize(size_t rows, size_t cols);

//- Change size and capacity in one shot.
//
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void resize(size_tuple size, size_tuple cap);
template<class ET2 = ENG, enable_if_resizable_t<ENG, ET2> = true>
void resize(size_t rows, size_t cols, size_t rowcap, size_t colcap);

//- Row and column operations.
//
void swap_columns(size_t i, size_t j);
void swap_rows(size_t i, size_t j);

private:

```

```

    engine_type    m_engine;    //- for exposition
};

```

Class template `matrix<ENG>` provides a representation of a matrix, with element type and storage management implemented by the engine type `ENG`. If the engine provides resizing, then this class will as well.

§ 6.3. Matrix Operation Traits

§ 6.3.1. Negation Traits

Class template `matrix_negation_traits<E1>` is an arithmetic traits type that performs a negation of a *MathObj* and returns the result in another *MathObj* having an implementation-defined engine type.

```

template<class E1>                                //- for exposition only; ba
struct matrix_negation_traits                      // not implemented
{
    using engine_type = ...;                      //- implementation-defined
    using result_type = MathObj<engine_type>;    //- appropriate MathObj ret

    static result_type negate(MathObj<E1> const& m);
};

template<class E1>
struct matrix_negation_traits<column_vector<E1>>;

template<class E1>
struct matrix_negation_traits<row_vector<E1>>;

template<class E1>
struct matrix_negation_traits<matrix<E1>>;

```

§ 6.3.2. Addition Traits

Class template `matrix_addition_traits<E1, E2>` is an arithmetic traits type that performs an addition of two compatible *MathObjs* and returns the result in a *MathObj* having an implementation-defined engine type.

```

template<class E1, class E2>                                //- for exposition only; ba
struct matrix_addition_traits                                // not implemented
{
    using engine_type = ...;                                //- implementation-defined
    using result_type = MathObj<engine_type>;               //- appropriate MathObj ret

    static result_type add(MathObj<E1> const& m1, MathObj<E2> const& m2);
};

template<class E1, class E2>
struct matrix_addition_traits<column_vector<E1>, column_vector<E2>>;

template<class E1, class E2>
struct matrix_addition_traits<row_vector<E1>, row_vector<E2>>;

template<class E1, class E2>
struct matrix_addition_traits<matrix<E1>, matrix<E2>>;

```

§ 6.3.3. Subtraction Traits

Class template `matrix_subtraction_traits<E1, E2>` is an arithmetic traits type that performs a subtraction of two compatible *MathObj*s and returns the result in a *MathObj* having an implementation-defined engine type.

```

template<class E1, class E2>                                //- for exposition only; ba
struct matrix_subtraction_traits                            // not implemented
{
    using engine_type = ...;                                //- implementation-defined
    using result_type = MathObj<engine_type>;              //- appropriate MathObj ret

    static result_type subtract(MathObj<E1> const& m1, MathObj<E2> const&
};

template<class E1, class E2>
struct matrix_subtraction_traits<column_vector<E1>, column_vector<E2>>;

template<class E1, class E2>
struct matrix_subtraction_traits<row_vector<E1>, row_vector<E2>>;

template<class E1, class E2>
struct matrix_subtraction_traits<matrix<E1>, matrix<E2>>;

```

§ 6.3.4. Multiplication Traits

Class template `matrix_multiplication_traits<E1, E2>` is an arithmetic traits type that performs the multiplication of two compatible *MathObjs* and returns the result in a *MathObj* having an implementation-defined engine type.

```

template<class E1, class E2>                                //- for exposition only; ba
struct matrix_multiplication_traits                          // not implemented
{
    using engine_type = ...;                                //- implementation-defined
    using result_type = MathObj<engine_type>;               //- appropriate MathObj ret

    static result_type multiply(MathObj<E1> const& m1, MathObj<E2> const&
};

template<class E1, class T2>
struct matrix_multiplication_traits<column_vector<E1>, T2>;

template<class E1, class T2>
struct matrix_multiplication_traits<row_vector<E1>, T2>;

template<class E1, class T2>
struct matrix_multiplication_traits<matrix<E1>, T2>;

template<class E1, class E2>
struct matrix_multiplication_traits<row_vector<E1>, column_vector<E2>>;

template<class E1, class E2>
struct matrix_multiplication_traits<column_vector<E1>, row_vector<E2>>;

template<class E1, class E2>
struct matrix_multiplication_traits<matrix<E1>, column_vector<E2>>;

template<class E1, class E2>
struct matrix_multiplication_traits<matrix<E1>, row_vector<E2>>;

template<class E1, class E2>
struct matrix_multiplication_traits<column_vector<E1>, matrix<E2>>;

template<class E1, class E2>
struct matrix_multiplication_traits<row_vector<E1>, matrix<E2>>;

template<class E1, class E2>
struct matrix_multiplication_traits<matrix<E1>, matrix<E2>>;

```


§ 6.4. Arithmetic Operators

§ 6.4.1. Negation Operator

The unary negation operators are provided to perform element-wise negation of a *MathObj* instance.

```
template<class E1>
auto operator -(column_vector<E1> const& m1);

template<class E1>
auto operator -(row_vector<E1> const& m1);

template<class E1>
auto operator -(matrix<E1> const& m1);
```

Function template `operator -(MathObj<E1> const&)` is equivalent to multiplying the argument by the scalar value of `static_cast<E1>(-1)`.

§ 6.4.2. Addition Operator

The following binary operators are provided to perform element-wise addition of two *MathObj* instances.

```
template<class E1, class E2>
auto operator +(column_vector<E1> const& m1, column_vector<E2> const& m2);

template<class E1, class E2>
auto operator +(row_vector<E1> const& m1, row_vector<E2> const& m2);

template<class E1, class E2>
auto operator +(matrix<E1> const& m1, matrix<E2> const& m2);
```

Function template `operator +(MathObj<E1> const &, MathObj<E2> const &)` performs addition between two *MathObj* instances of identical dimension.

§ 6.4.3. Subtraction Operator

The following binary operators are provided to perform element-wise subtraction of two *MathObj* instances.

```
template<class E1, class E2>
auto operator -(column_vector<E1> const& m1, column_vector<E2> const& m2);

template<class E1, class E2>
auto operator -(row_vector<E1> const& m1, row_vector<E2> const& m2);

template<class E1, class E2>
auto operator -(matrix<E1> const& m1, matrix<E2> const& m2);
```

Function template `operator -(MathObj<E1> const&, MathObj<E2> const&)` performs subtraction between two `MathObj` instances of identical dimension.

§ 6.4.4. Multiplication Operator

The following binary operators are provided to perform multiplication of two *MathObj* instances.

```

template<class E1, class E2>
auto operator *(column_vector<E1> const& cv, E2 s);

template<class E1, class E2>
auto operator *(E1 s, column_vector<E2> const& cv);

template<class E1, class E2>
auto operator *(row_vector<E1> const& rv, E2 s);

template<class E1, class E2>
auto operator *(E1 s, row_vector<E2> const& rv);

template<class E1, class E2>
auto operator *(matrix<E1> const& m, E2 s);

template<class E1, class E2>
auto operator *(E1 s, matrix<E2> const& m);

template<class E1, class E2>
auto operator *(row_vector<E1> const& rv, column_vector<E2> const& cv);

template<class E1, class E2>
auto operator *(column_vector<E1> const& cv, row_vector<E2> const& rv);

template<class E1, class E2>
auto operator *(matrix<E1> const& m, column_vector<E2> const& cv);

template<class E1, class E2>
auto operator *(matrix<E1> const& m, row_vector<E2> const& rv);

template<class E1, class E2>
auto operator *(column_vector<E1> const& cv, matrix<E2> const& m);

template<class E1, class E2>
auto operator *(row_vector<E1> const& rv, matrix<E2> const& m);

template<class E1, class E2>
auto operator *(matrix<E1> const& m1, matrix<E2> const& m2);

```

Function templates `operator *(column_vector<E1> const&, E2)`, `operator *(E1, column_vector<E2> const&)`, `operator *(row_vector<E1> const&, E2)`, `operator *`

(`E1`, `row_vector<E2> const&`), `operator *(matrix<E1> const&, E2)`, and `operator *(E1, matrix<E2> const&)` perform multiplication of a *MathObj* instance and a scalar value.

Function templates `operator *(row_vector<E1> const&, column_vector<E2> const&)` and `operator *(column_vector<E1> const&, row_vector<E2> const&)` perform the inner product and outer product, respectively.

Function templates `operator *(matrix<E1> const&, column_vector<E2> const&)`, `operator *(matrix<E1> const&, row_vector<E2> const&)`, `operator *(column_vector<E1> const&, matrix<E2> const&)`, `operator *(row_vector<E1> const&, matrix<E2> const&)`, and `operator *(matrix<E1> const&, matrix<E2> const&)` perform matrix multiplication of *MathObjs*.

§ 6.5. Type Aliases

These type aliases provide useful shorthand symbols for the most common compositions of types.

```
template<class T, class A = std::allocator<T>>
using dyn_col_vector = column_vector<dyn_matrix_engine<T, A>>;

template<class T, class A = std::allocator<T>>
using dyn_row_vector = row_vector<dyn_matrix_engine<T, A>>;

template<class T, class A = std::allocator<T>>
using dyn_matrix = matrix<dyn_matrix_engine<T, A>>;

template<class T, size_t R>
using fs_col_vector = column_vector<fs_matrix_engine<T, R, 1>>;

template<class T, size_t C>
using fs_row_vector = row_vector<fs_matrix_engine<T, 1, C>>;

template<class T, size_t R, size_t C>
using fs_matrix = matrix<fs_matrix_engine<T, R, C>>;
```